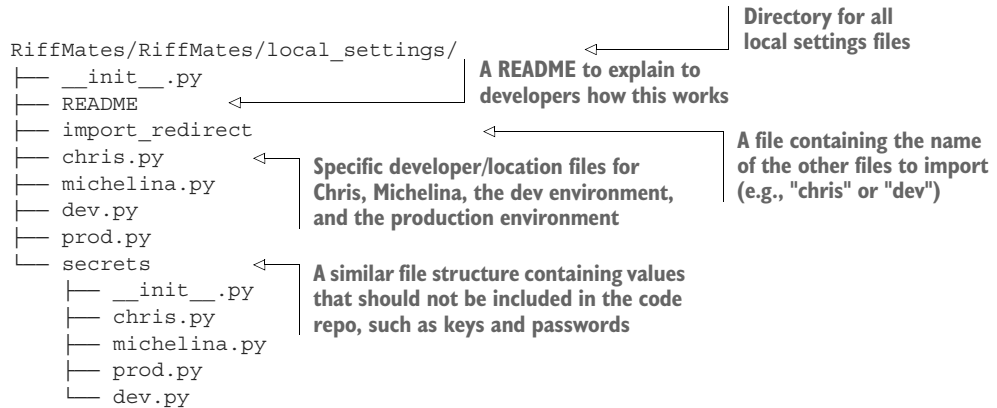


This is probably overkill for a small project, but when you have multiple environments and a team of developers, it allows you to set configuration values specific to both a deployment target and, if necessary, a developer. For example, if we were working together and you wanted to use the Django Debug Toolbar but I didn't, this would be possible because we'd both have our own configuration file. An example file tree looks like this:



The `import_redirect` file and the `secrets` directory aren't included in your code repo. Adding the following to `.gitignore` ensures they won't be committed accidentally:

```
# .gitignore
...
RiffMates/local_settings/import_redirect
RiffMates/local_settings/secrets/
```

The code in `settings.py` that does the importation work can be seen in the following listing.

Listing B.1 Code for dynamically importing local settings and secret modules

```
# RiffMates/RiffMates/settings.py
...
# ---- local settings import overrides

# detect if in dev server mode
import sys
DEV_SERVER = (len(sys.argv) > 1 and \
              sys.argv[1] == 'runserver')

redirect = (BASE_DIR /
            'RiffMates/local_settings/import_redirect')
with open(redirect, 'r') as f:
    import_file = f.read().strip()

action = 'from RiffMates.local_settings.%s import *' \
        % import_file
```

Annotations:

- Detect if you're running the development server. (points to `DEV_SERVER`)
- The file containing the name of other files to import (points to `import_redirect`)
- Read `import_redirect`, and get the name of the other imports. (points to `import_file`)
- Build the Python code that imports all values from the requested local settings file. (points to `action`)